

Transforma texto en embeddings

Jenifer Cárdenas Aguilera

Facultad de Ingeniería, Universidad de Cundinamarca

Especialización en Inteligencia Artificial

Procesamiento de Lenguaje Natural

Septiembre de 2026

Introducción

Este informe documenta el trabajo práctico “Transforma texto en embeddings”, desarrollado a partir de la serie *NLP Zero to Hero* de TensorFlow (Moroney, 2019). El notebook adjunto recorre seis partes: tokenización, secuencias y padding, embeddings para clasificar sentimiento, redes neuronales recurrentes (RNN), LSTM, y generación de texto.

El notebook completo, con el código ejecutado, se encuentra publicado en el siguiente repositorio de GitHub: https://github.com/Fefy39/transforma_texto_en_embeddings

Parte 1 — Tokenización

La tokenización convierte cada palabra distinta de un texto en un número entero, usando la clase `Tokenizer` de Keras.

```
frases = [
    'Me encanta mi perro',
    'Me encanta mi gato',
    'Tú amas a mi perro!',
    '¿Crees que mi perro es increíble?',
    'Amo a mis mascotas y a mi familia',
    'Quiero tener dos mascotas, un perro y un gato',
]

tokenizer = Tokenizer(num_words=50, oov_token="<OOV>")
tokenizer.fit_on_texts(frases)
word_index = tokenizer.word_index
secuencias = tokenizer.texts_to_sequences(frases)
```

El diccionario palabra → número obtenido fue:

```
{'<OOV>': 1, 'mi': 2, 'perro': 3, 'a': 4, 'me': 5, 'encanta': 6, 'gato': 7,
 'mascotas': 8, 'y': 9, 'un': 10, 'tú': 11, 'amas': 12, '¿crees': 13, 'que': 14,
 'es': 15, 'increíble': 16, 'amo': 17, 'mis': 18, 'familia': 19, 'quiero': 20,
 'tener': 21, 'dos': 22}
```

Al tokenizar frases con palabras nuevas (“Mi perro se come mi tarea”, “Mi gato es amigo de mi perro”), las palabras no vistas durante el entrenamiento se reemplazan por el número asignado al token `<OOV>` (1):

```
[[2, 3, 1, 1, 2, 1], [2, 7, 15, 1, 1, 2, 3]]
```

Parte 2 — Secuencias y padding

Como las frases tienen longitudes distintas, se usa `pad_sequences` para que todas midan lo mismo, rellenando con ceros o recortando.

```
secuencias_padded = pad_sequences(secuencias, padding='pre', maxlen=10)
secuencias_padded_fijo = pad_sequences(secuencias, padding='post', maxlen=5,
                                       truncating='post')
```

Con relleno automático (`maxlen=10`, padding al inicio):

```
[[ 0  0  0  0  0  0  5  6  2  3]
 [ 0  0  0  0  0  0  5  6  2  7]
 [ 0  0  0  0  0 11 12  4  2  3]
 [ 0  0  0  0 13 14  2  3 15 16]
 [ 0  0 17  4 18  8  9  4  2 19]
 [ 0 20 21 22  8 10  3  9 10  7]]
```

Con longitud fija (`maxlen=5`, padding y recorte al final):

```
[[ 5  6  2  3  0]
 [ 5  6  2  7  0]
 [11 12  4  2  3]
 [13 14  2  3 15]
 [17  4 18  8  9]
 [20 21 22  8 10]]
```

Parte 3 — De números a embeddings

Los números de la tokenización son solo etiquetas; no existe una relación matemática real entre ellos. Un **embedding** representa cada palabra con un vector de varias dimensiones que se ajusta durante el entrenamiento, de modo que palabras usadas en contextos parecidos quedan cerca en ese espacio.

Se construyó un corpus sintético de 180 reseñas en español (90 positivas y 90 negativas), combinando 6 temas con 15 valoraciones de cada tipo, y se dividió en 135 frases de entrenamiento y 45 de prueba.

```

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(6, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

```

El modelo se entrenó con **EarlyStopping** (40 épocas máximo) sobre las 135 reseñas de entrenamiento. Resultado:

Métrica	Entrenamiento	Validación
Exactitud (accuracy)	0.956	1.000

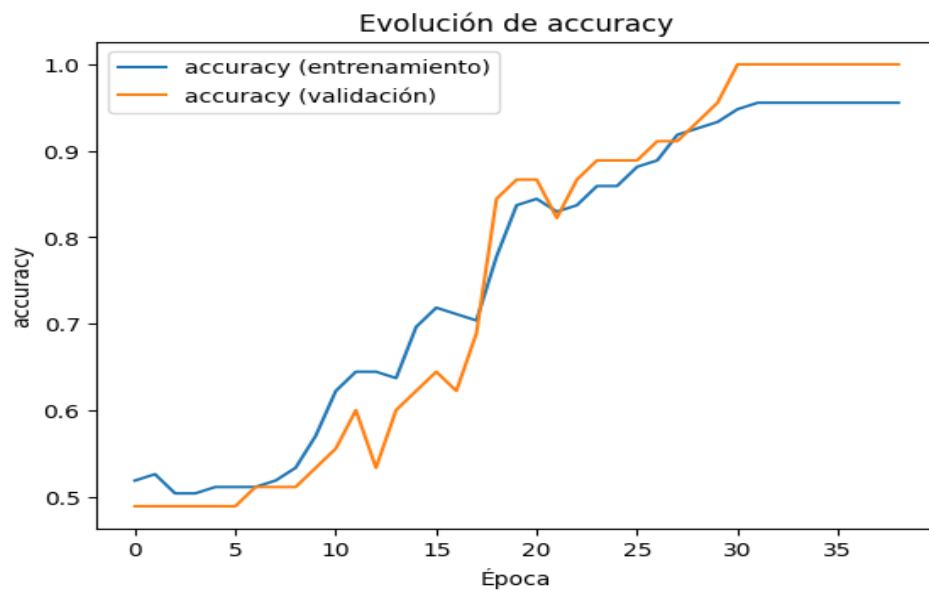


Figura 1. Evolución de la exactitud (accuracy) durante el entrenamiento.

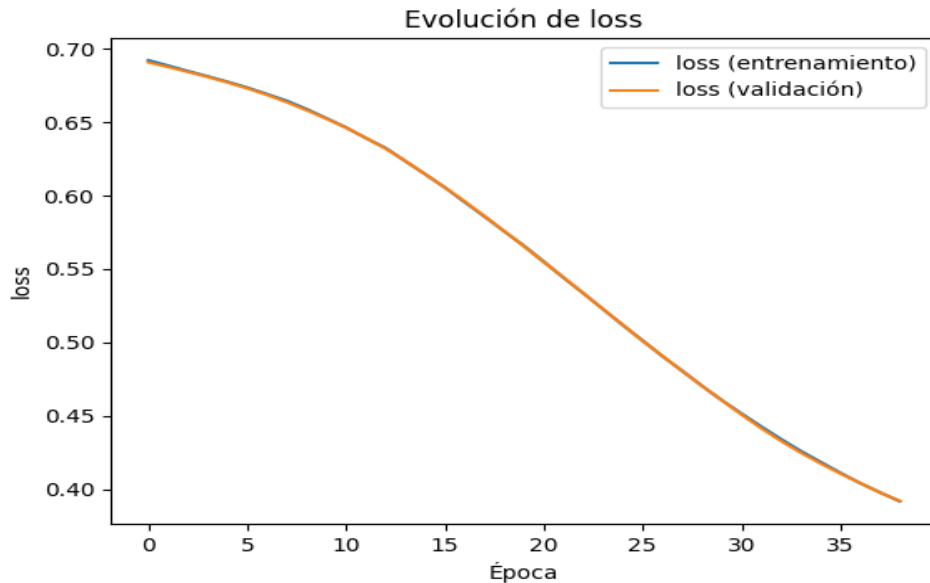


Figura 2. Evolución de la pérdida (loss) durante el entrenamiento.

Parte 4 — Redes neuronales recurrentes (RNN)

Una RNN procesa la frase palabra por palabra, en orden, y mantiene un estado interno que se va actualizando con cada palabra nueva. Así, la representación final de la frase sí depende del orden en que aparecieron las palabras (algo que el modelo de la Parte 3, al promediar los embeddings, no tiene en cuenta).

```
model_rnn = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.Bidirectional(tf.keras.layers.SimpleRNN(16)),
    tf.keras.layers.Dense(6, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model_rnn.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model_rnn.summary()
```

Parte 5 — LSTM (Long Short-Term Memory)

Las RNN simples tienen un problema práctico: en frases largas, la información de las primeras palabras tiende a diluirse antes de llegar al final (el problema del gradiente que desaparece). Las celdas **LSTM** agregan “puertas” que deciden qué información conservar, cuál

olvidar y cuál dejar pasar, lo que ayuda a recordar mejor relaciones separadas por varias palabras.

En Keras basta con cambiar SimpleRNN por LSTM.

```
model_lstm = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(16)),
    tf.keras.layers.Dense(6, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model_lstm.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
history_lstm = model_lstm.fit(train_padded, train_labels, epochs=num_epochs,
                              validation_data=(test_padded, test_labels),
                              callbacks=[early_stop], verbose=0)
```

Resultado: accuracy entrenamiento = 0.933, accuracy validación = 0.933.

Parte 6 — Generar texto: de clasificar a crear

En esta última parte se cambia de tarea: en lugar de predecir una etiqueta, se entrena un modelo para predecir la siguiente palabra de una frase, usando un corpus original de ocho versos cortos. Por cada línea se generan todas sus secuencias de n-gramas, y se entrena una red Embedding + LSTM bidireccional + Dense(softmax) sobre las 44 palabras distintas del vocabulario.

```
model_gen = tf.keras.Sequential([
    tf.keras.layers.Embedding(total_palabras, 240, input_length=max_seq_len - 1),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(150)),
    tf.keras.layers.Dense(total_palabras, activation='softmax')
])

adam = tf.keras.optimizers.Adam(learning_rate=0.01)
model_gen.compile(loss='categorical_crossentropy', optimizer=adam,
                  metrics=['accuracy'])
history_gen = model_gen.fit(X, y, epochs=100, verbose=0)
```

Accuracy final prediciendo la siguiente palabra: **0.904**. Al generar texto nuevo a partir de una semilla se obtuvo:

```
generar_texto('el rio corre', 6, ...)
-> 'el rio corre entre las piedras del camino camino'
```

```
generar_texto('las estrellas', 6, ...)
-> 'las estrellas brillan sobre el mar en calma'

generar_texto('el viento', 8, ...)
-> 'el viento canta bajo la luna llena llena llena llena'
```

Conclusiones

- La tokenización convierte palabras en números; por sí sola no captura significado.
- El padding estandariza el largo de las secuencias para poder entrenar en lotes (batches).
- La capa Embedding aprende, junto con el resto de la red, una representación vectorial de cada palabra en la que la cercanía geométrica refleja cercanía semántica.

Referencias

- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 1*](#) [Video]. YouTube.
- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 2*](#) [Video]. YouTube.
- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 3*](#) [Video]. YouTube.
- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 4*](#) [Video]. YouTube.
- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 5*](#) [Video]. YouTube.
- Moroney, L. (2019). [*Natural language processing zero to hero – Parte 6*](#) [Video]. YouTube.
- TensorFlow. (s.f.). [*tf.keras.preprocessing.text.Tokenizer*](#). Documentación de la API de Keras.
- TensorFlow. (s.f.). [*tf.keras.layers.Embedding*](#). Documentación de la API de Keras.
- TensorFlow. (s.f.). [*Embedding projector*](#).